0x40000000 不是魔法数：把指针放到寄存器上

嵌入式 Linux 手记　2026-10-23　公众号文章 · 原链接 mp.weixin.qq.com

先看一段"不像 C 程序"的 C 程序。这是 STM32F4 单片机（Cortex-M4 内核）点亮 PA5 上那颗 LED 的核心代码，SPL/HAL 库一概不用：

```c
#include <stdint.h>

#define RCC_BASE      0x40023800UL
#define GPIOA_BASE    0x40020000UL

volatileuint32_t *rcc_ahb1en = (volatileuint32_t *)(RCC_BASE + 0x30);
volatileuint32_t *gpio_moder = (volatileuint32_t *)(GPIOA_BASE + 0x00);
volatileuint32_t *gpio_bsrr  = (volatileuint32_t *)(GPIOA_BASE + 0x18);

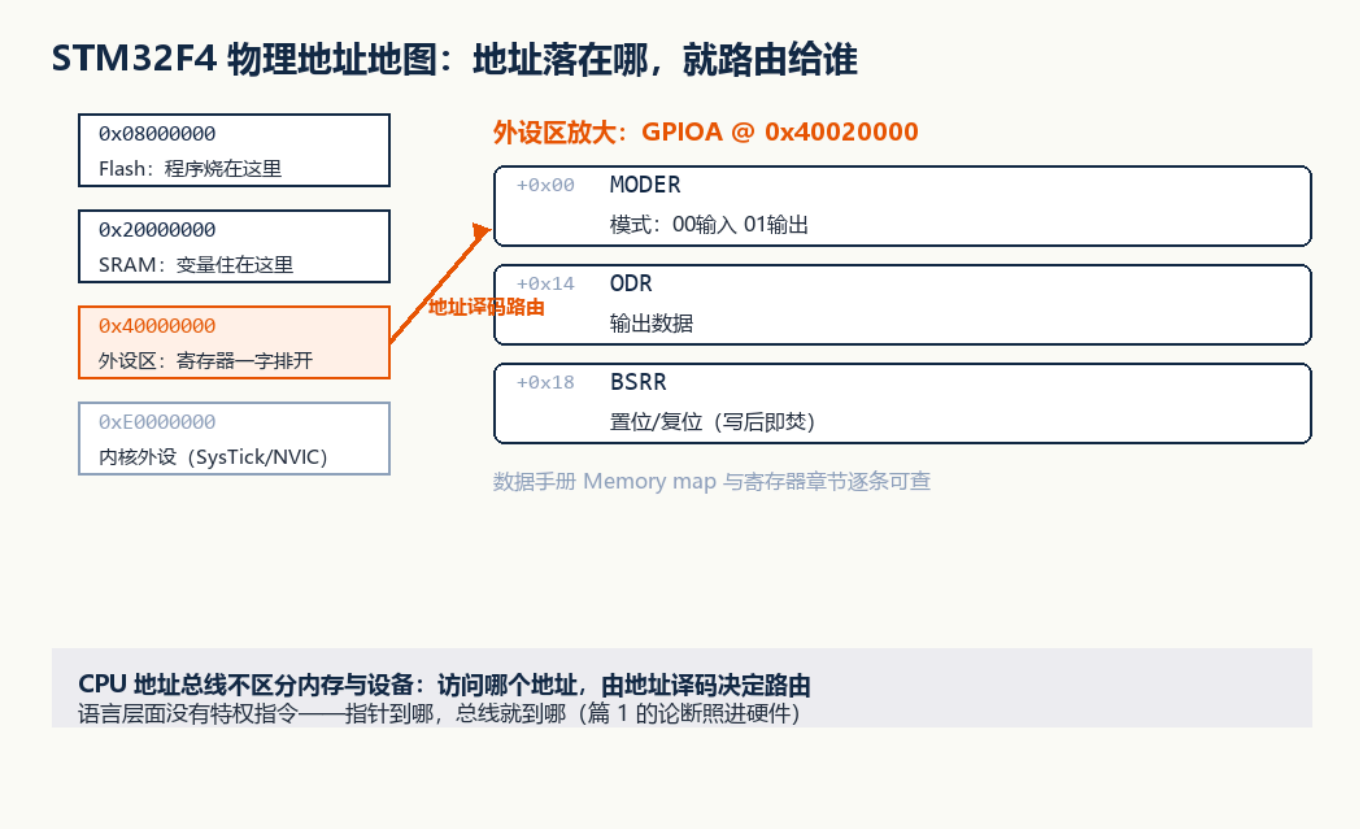
*rcc_ahb1en |= (1UL << 0);                          /* 打开 GPIOA 的时钟 */
*gpio_moder = (*gpio_moder & ~(3UL << 10)) | (1UL << 10);   /* PA5 设为输出 */
*gpio_bsrr  = (1UL << 5);                           /* PA5 置高，LED 亮 */
```

没有 main 以外的依赖，没有 malloc，没有 printf，没有操作系统：三条赋值，就是驱动。那些 `0x40020000` 一类的数字，不是魔法，是数据手册存储器映射章节里印着的物理地址。这一篇把指针深水区的最后一块拼图装上：指针离开内存，直接放到硬件上。

一、地址即设备：MMIO 的译码路由

`0x40020000` 这个地址上没有 RAM，为什么解引用它能"操作设备"？先看这颗芯片的物理地址地图（数据手册的 Memory map 章节）：`0x08000000` 起是 Flash，程序烧在这里；`0x20000000` 起是 SRAM，变量住在这里；`0x40000000` 起一大段是外设区，GPIO、定时器、串口的控制寄存器一字排开，`GPIOA` 恰好住在 `0x40020000`。

这套设计叫 MMIO（Memory-Mapped I/O，内存映射 I/O）：把外设寄存器挂进统一的内存地址空间，CPU 地址总线发出一个地址时并不区分"这是内存还是设备"，区分由地址译码完成：地址落在哪一段，就被路由给哪块硬件。访问 `0x40020000` 时，译码器把这次读写交给 GPIOA，而不是内存控制器；写入的每一位直接变成引脚电平或电路配置。



图：物理地址地图一角。地址落在哪一段，译码器就把这次访问路由给谁；外设区放大后，GPIOA 的每个寄存器都有手册可查的偏移。

C 语言里指针的本质就是地址值，地址能到的地方，地址总线就能到。对编译器而言，`*gpio_bsrr = (1UL << 5)` 与给一个普通 int 赋值没有区别；区别全在那条地址落在哪。对地址空间的每一段而言，语言层面没有特权指令，只有访问哪个格子：这就是裸机 C 与桌面 C 的第一处分野。

二、volatile 不可省略：与编译器的正面交锋

把三处声明里的 `volatile` 删掉再编译，程序多半还是亮的，这才是最危险的瞬间。编译器看 `*rcc_ahb1en |= (1UL << 0)`，认为这不过是"读一个普通内存、改一位、写回"，按 as-if 规则它有权做两件"等价"的事：把对同一普通地址的连续写合并成一次，或把"写了没人再读"的访问整个删除。对 RAM 这样做天经地义；对硬件寄存器做，时钟可能没开、配置可能没生效，程序在硬件层面已经错了，编译器却自认无辜。

更深一层，普通内存还背着一条"写入后读出仍是原值"的默认假设。寄存器不守约：状态寄存器随时可能被硬件改写（时钟就绪标志），`BSRR` 这类寄存器写后即焚（写 1 置位、写 0 无效，读回无意义）。volatile 的语义在 C11 §6.7.3，对其访问的严格求值构成 §5.1.2.3 定义的可观察行为，编译器不得删除、不得合并、不得重排，每一次 `*p` 都必须真的变成一条访存指令落到总线

上。篇 8 讲过 UB 折叠，这里的对抗同源：你用关键字把"这个格子不归 as-if 管"写进声明，编译器才收手。

类型怎么写最容易错，正好用篇 2 的四要素拆一遍：

```
volatile uint32_t *p;    /* p 是指针，指向 volatile uint32_t：每次经 p 的访问都不可优化 */
uint32_t * volatile q;   /* q 本身 volatile，指向普通 uint32_t：指针变量不可优化，访问照样会被合并
*/
```

一对括号之差，寄存器指针要的是前者：volatile 修饰的是被指向的对象（每次访问是真的），不是指针自己。SDK 头文件里满天飞的 `__IO uint32_t`，展开后正是 `volatile uint32_t`。

三、uintptr_t 与正确的类型姿势

整数转指针、指针转整数，标准怎么管？§6.3.2.3：整数可以转换为指针（结果为实现定义），指针也能转整数。这两步是裸机开发的日常，但想可移植，中间人应该是 C99 的 `uintptr_t`（§7.20.1.4）：承诺能装下 `void*` 的无符号整数类型，32 位机 4 字节、64 位机 8 字节，自动跟随。

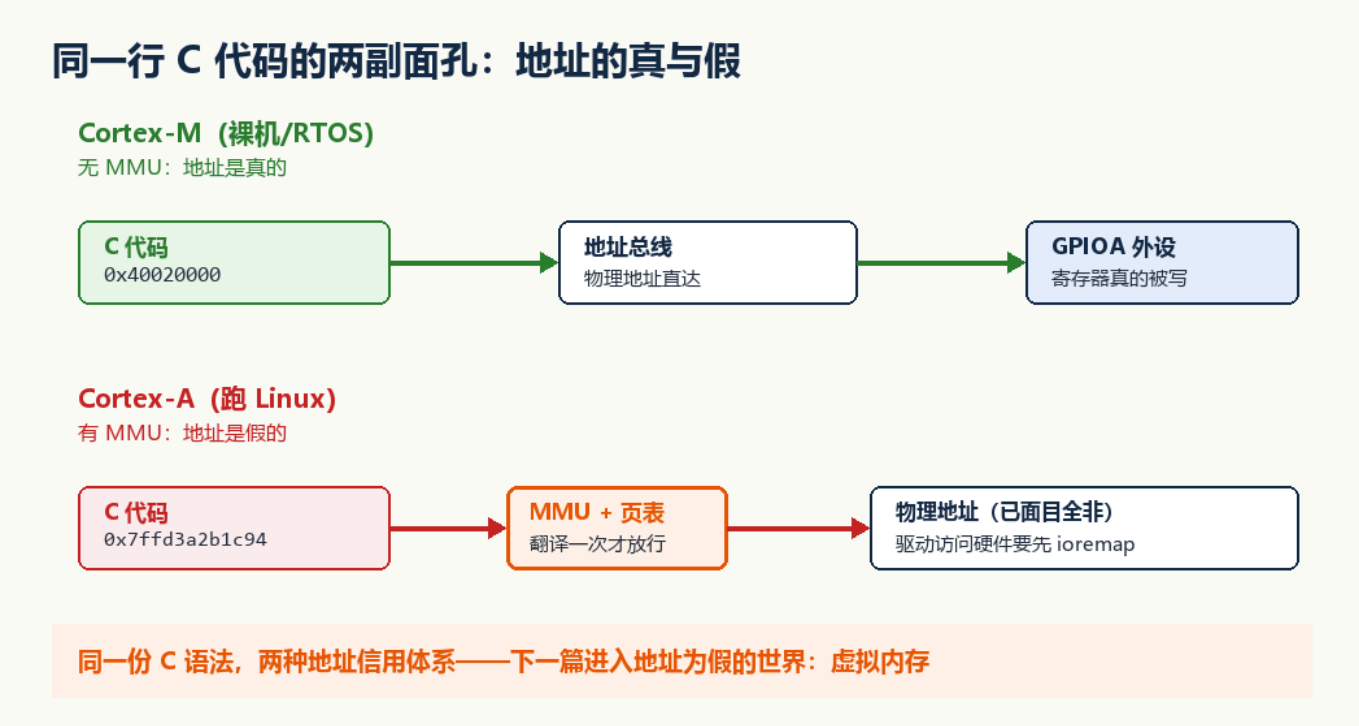
为什么不用 `unsigned int` 接地址？LP64 平台（Linux x86-64）上 `unsigned int` 只有 4 字节，8 字节地址塞进去即截断，往返一次地址就残了。寄存器指针的完整姿势因此有三件套：地址用 `uintptr_t` 或带 `UL` 后缀的字面量（保证 32 位无符号），宽度用 `uint32_t` 而非 `unsigned long`（后者在 LP64 下是 8 字节，寄存器只有 4 字节宽），再加 volatile 修饰被指向对象。`uint32_t` 这类固定宽度类型存在的意义，在跨 32/64 位的工程里才真正显形。

还有一处只有硬件现场才有的坑：`*gpio_moder = (*gpio_moder & ~(3UL << 10)) | (1UL << 10)` 这句"读改写"是三条动作（读、算、写），不是原子的。若中断服务程序在中间也改这个寄存器，主程序写回时会把中断的修改覆盖掉。桌面编程里这叫竞态，单片机里它真的会发生：对策是关中断保护临界段，或用硬件的原子置位/复位寄存器（`BSRR` 之所以存在，就是给"不打断别人的置位"用的）。指针送到硬件上之后，并发的世界也跟着到了。

四、同一行代码的两副面孔

最后回答篇 14 留下的问题：这算不算"虚拟内存的对立面"？算，但对立得很有层次。

Cortex-M 这类单片机没有 MMU（至多配 MPU 做权限保护），C 程序发出的地址直接走上地址总线，`0x40020000` 是物理地址，译码路由直达外设，地址是真的。Cortex-A 芯片跑 Linux，MMU + 页表把每个程序发出的地址先翻译一道，程序看到的 `0x7ffd3a2b1c94` 只是虚拟地址，落到总线前已经面目全非，地址是假的。同一份 C 语法，两种世界两副面孔。



图：同一行 C 代码的两副面孔。Cortex-M 上地址直达硬件；Cortex-A 上地址先经 MMU 翻译，驱动必须 ioremap 后才能摸到硬件。

Linux 驱动因此多一道手续：拿到数据手册上的物理地址后，不能直接 `(volatile uint32_t *)0x40020000` 解引用，必须先经 `ioremap` 把物理地址映射进内核的虚拟空间，再用映射后的指针访问。语言没变，变的是地址的信用体系。写裸机时指针即坐标，写内核驱动时指针是"坐标的坐标"：篇 2 拆过的类型位置、篇 3 拆过的步长、篇 8 拆过的悬空、本篇的 volatile 与 uintptr_t，全部在这里会师。

写在最后

1. MMIO 把外设寄存器挂进内存地址空间，访问哪个地址由地址译码路由决定；`0x40020000` 是数据手册印着的坐标，不是魔法数。
2. volatile 是写给编译器的停战协议（C11 §6.7.3）：访问是可观察行为，删除/合并/重排全部禁止；它修饰的是被指向的对象，`volatile uint32_t *p` 而非 `uint32_t * volatile p`。
3. 地址往返用 `uintptr_t`（§7.20.1.4），宽度自动匹配指针；`unsigned int` 在 64 位下截断地址；寄存器宽度用 `uint32_t` 这类固定宽度类型。

4. 读改写三步不是原子，中断并发会覆盖修改；BSRR 这类置位/复位寄存器就是硬件给的原子写通道。
5. 地址的真假由平台有无 MMU 决定：Cortex-M 地址即物理，Cortex-A 上 Linux 驱动必须 `ioremap` 后才能访问。

至此，指针系列十五篇收束：从"数组名不是指针"一路走到寄存器边缘。下一步进入内存与底层的世界：C 程序打印出的那些 `0x7ffd...` 地址，到底是谁的手笔？下一篇《你打印的地址是假的：虚拟内存到底替你做了什么》，把地址信用体系的另一半补完。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。